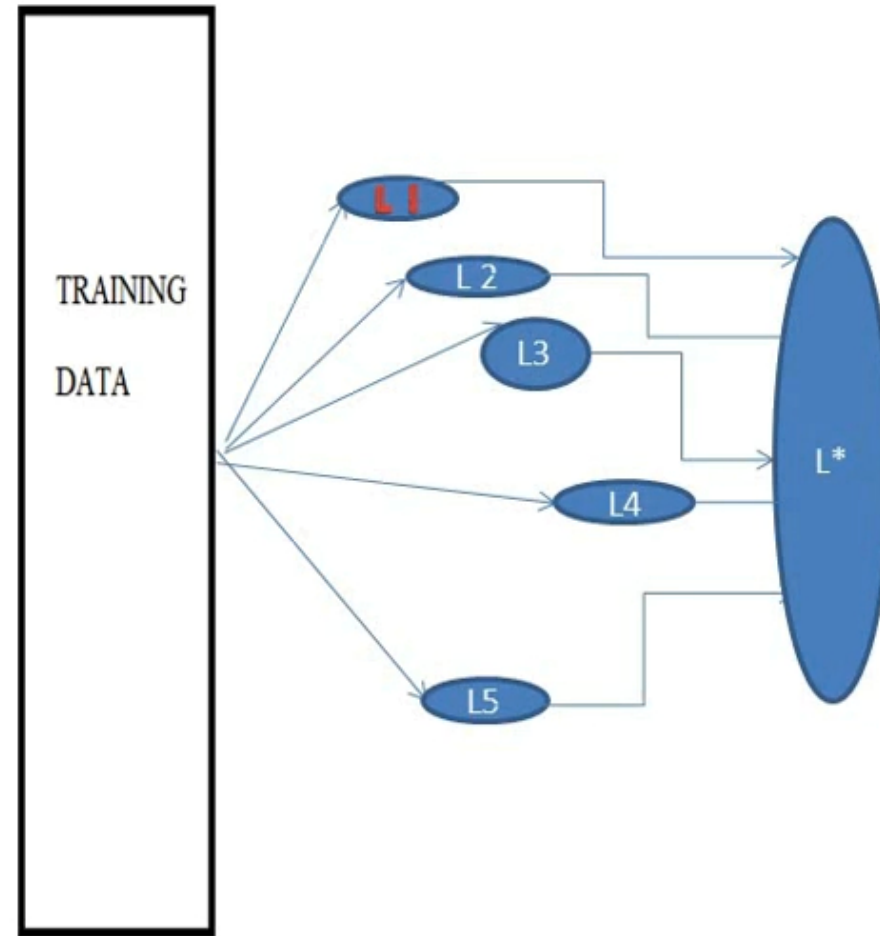


Ensemble Classifiers / Learning

Combining Multiple Learners – Ensemble Method

- ▷ In general there are several algorithms for learning the same task.
- ▷ Though these are generally successful, no one single algorithm is always the most accurate.
- ▷ The No Free Lunch Theorem states that there is no single learning algorithm that in any domain always induces the most accurate learner.
- ▷ The usual approach is to try many and choose the one that performs the best on a separate validation set.
- ▷ We may combine multiple learning algorithms or same algorithms with different hyperparameters as classifiers.
- ▷ Here we discuss a models composed of multiple learners that complement each other so that by combining them, we attain higher accuracy.



Why combine many learners together?

There are several reasons why a single learner may not produce accurate results.

- Each learning algorithm carries with it a set of assumptions.
 - This leads to error if the assumptions do not hold.
 - We cannot be fully sure whether the assumptions are true in a particular situation.
- ▷ Learning is an ill-posed problem . With finite data, each algorithm may converge to a different solution and may fail in certain circumstances.
- ▷ The performance of a learner may be fine-tuned to get the highest possible accuracy on a validation set.
 - But this fine-tuning is a complex task and still there are instances on which even the best learner is not accurate enough.
- ▷ It has been proved that there is no single learning algorithm that always produces the most accurate output.

Ways to achieve diversity

- ▷ When many learning algorithms are combined, the individual algorithms in the collection are called the **base learners** of the collection.
- ▷ There are several different ways for selecting the base learners.
 1. Use different learning algorithms
 2. Use the same algorithm with different hyper parameters
 3. Use different representations of the input object
 4. Use different training sets to train different base-learners

Ways to achieve diversity

- ▷ When many learning algorithms are combined, the individual algorithms in the collection are called the **base learners** of the collection.
- ▷ There are several different ways for selecting the base learners.
 1. Use different learning algorithms
 2. Use the same algorithm with different hyper parameters
 3. Use different representations of the input object
 4. Use different training sets to train different base-learners

Model Combination Schemes

▷ Voting - This is the simplest procedure for combining the outcomes of several learning algorithms

1. Binary classification problem

- ▷ Consider a binary classification problem with class labels -1 and $+1$.
- ▷ Let there be L base learners and let x be a test instance.
- ▷ Each of the base learners will assign a class label to x .
- ▷ If the class label assigned is $+1$, we say that the learner votes for $+1$ and that the label $+1$ gets a vote.
- ▷ The number of votes obtained by the class labels when the different base learners are applied is counted.
- ▷ In the voting scheme for combining the learners, the label which gets the majority votes is assigned to x .

Model Combination Schemes

2. Multi-class classification problem

- ▷ Let there be n class labels $C_1, C_2, \dots, C_n$.
- ▷ Let x be a test instance and let there be L base learners.
- ▷ Here also, each of the base learners will assign a class label to x and when a class label is assigned a label, the label gets a vote.
- ▷ In the voting scheme, the class label which gets the maximum number of votes is assigned to x .

Why use ensemble methods?

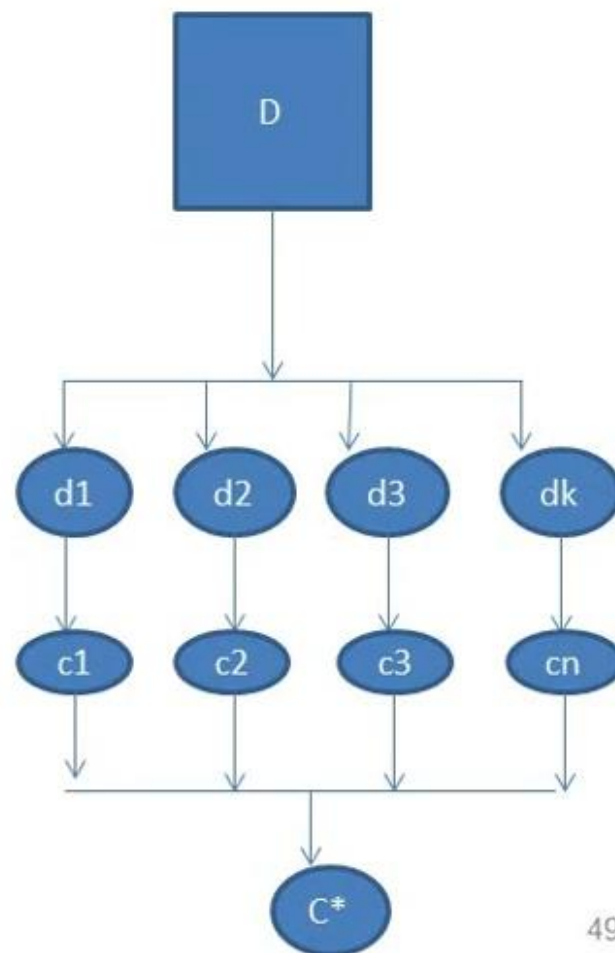
- Better accuracy (Low error)
- Higher consistency (Avoids overfitting)
- Reduces bias and variance errors

Homogenous Ensembles

- Use a **single, arbitrary learning algorithm** but **manipulate training data** to make it learn multiple models.
 - $\text{Data1} \neq \text{Data2} \neq \dots \neq \text{Data } m$
 - $\text{Learner1} = \text{Learner2} = \dots = \text{Learner } m$
- Different methods for changing training data:
 - Bagging: Resample training data
 - Boosting: Reweight training data
 - Random Forest.

Bagging – Bootstrap Aggregation

- Bagging is a voting method whereby base-learners are made different by training them over slightly different training sets.
- Generating L slightly different samples from a given sample is done by bootstrap, where given a training set X of size N , we draw N instances randomly from X with replacement
- Because sampling is done with replacement, it is possible that some instances are drawn more than once and that certain instances are not drawn at all.
- When this is done to generate L samples X_j , $j = 1, \dots, L$, these samples are similar because they are all drawn from the same original sample, but they are also slightly different due to chance.
- The base-learners are trained with these L samples X_j .



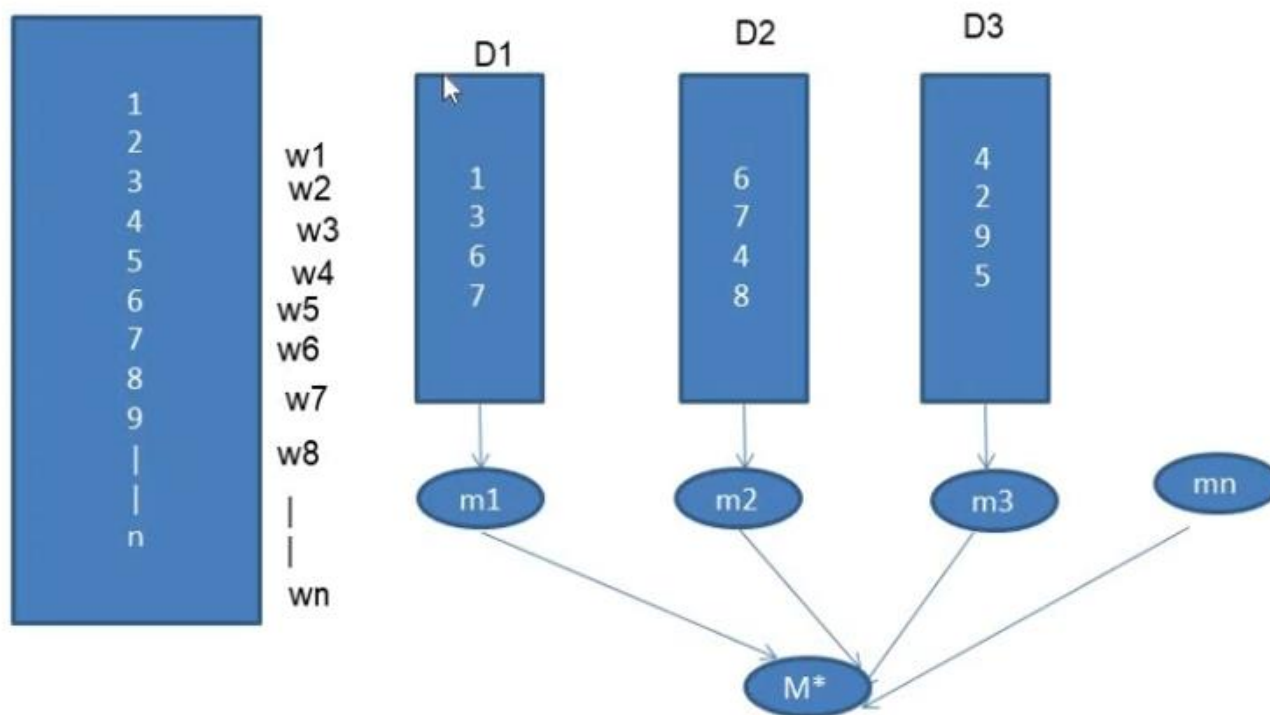
Bagging – Bootstrap Aggregation

- ▶ A learning algorithm is an *unstable algorithm*, if small changes in the training set causes a large difference in the generated learner.
- ▶ Algorithms such as decision trees and multilayer perceptrons are unstable.
- ▶ If we have an Initial dataset, D , Draw random samples from D with replacement (some instances repeated and some not).
- ▶ $D_1, D_2, \dots, D_k$ are subsets drawn from original dataset.
- ▶ Each of the subsets uses L base learners to train different models using an unstable learning procedure.
- ▶ Average of outputs of all models are taken.
- ▶ Bagging can be used both for classification and regression.
- ▶ In the case of regression, to be more robust, we can take the median instead of the average when combining predictions.

Boosting

- ▶ An iterative procedure
- ▶ The idea of boosting is to train weak learners sequentially, each trying to correct its predecessor.
- ▶ Finally, it combines the outputs from weak learner and creates a strong learner which eventually improves the prediction power of the model.
- ▶ Boosting pays higher focus on examples which are mis-classified or have higher errors by preceding weak rules.

Boosting



Boosting Procedure

- ▶ **Step 1:** The base learner takes all the distributions and assign equal weight or attention to each observation.
- ▶ **Step 2:** If there is any prediction error caused by first base learning algorithm, then we increase the weight of the observations having prediction error. Then, we apply the next base learning algorithm.
- ▶ **Step 3:** Iterate Step 2 till the limit of base learning algorithm is reached or higher accuracy is achieved.

Boosting

The original boosting method that combined three weak learners

- Let d_1, d_2, d_3 be three learning algorithms for a particular task. Let a large training set X be given.
- We randomly divide X into three sets, say X_1, X_2, X_3 .
- We use X_1 and train d_1 .
- We then take X_2 and feed it to d_1 .
- We take all instances misclassified by d_1 and also as many instances on which d_1 is correct from X_2 , and these together form the training set of d_2 .
- We then take X_3 and feed it to d_1 and d_2 .
- The instances on which d_1 and d_2 disagree form the training set of d_3 .
- During testing, given an instance, we give it to d_1 and d_2 if they agree, that is the response; otherwise the response of d_3 is taken as the output.

Bagging Vs Boosting

- Data Partition

- In Bagging data partition is random. In boosting, mis-classified data is given higher importance.

- Goal

- In bagging goal is reduce the variance in model. In boosting we aim to increase the prediction accuracy of the model.

- Method

- Use random subspace in bagging. Uses gradient descent method.

- Function

- Bagging uses weighted or average function. Boosting uses a weighted majority vote function.

Random Forest

- Ensemble method specifically designed for decision tree classifiers
- Random Forests grows many trees
 - Ensemble of unpruned decision trees
 - Each base classifier classifies a “new” vector of attributes from the original data
 - Final result on classifying a new instance: voting. Forest chooses the classification result having the most votes (over all the trees in the forest)

Algorithm for Random Forest

1. The random forests algorithm generates many classification trees. Each tree is generated as follows:
 - (a) If the number of examples in the training set is N , take a sample of N examples at random - but with replacement, from the original data. This sample will be the training set for generating the tree.
 - (b) If there are M input variables, a number m is specified such that at each node, m variables are selected at random out of the M and the best split on these m is used to split the node. The value of m is held constant during the generation of the various trees in the forest.
 - (c) Each tree is grown to the largest extent possible.
2. To classify a new object from an input vector, put the input vector down each of the trees in the forest. Each tree gives a classification, and we say the tree “votes” for that class. The forest chooses the classification

Random Forest Classifier

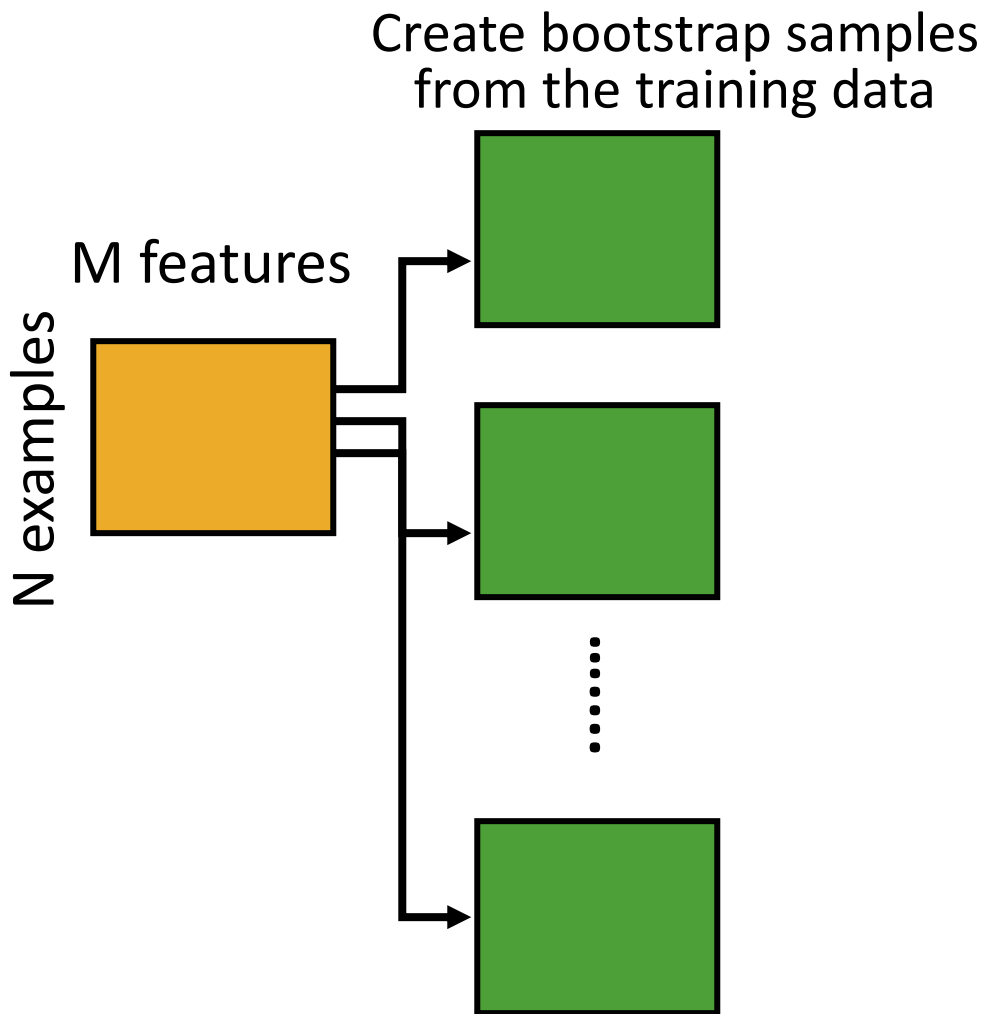
Training Data

N examples

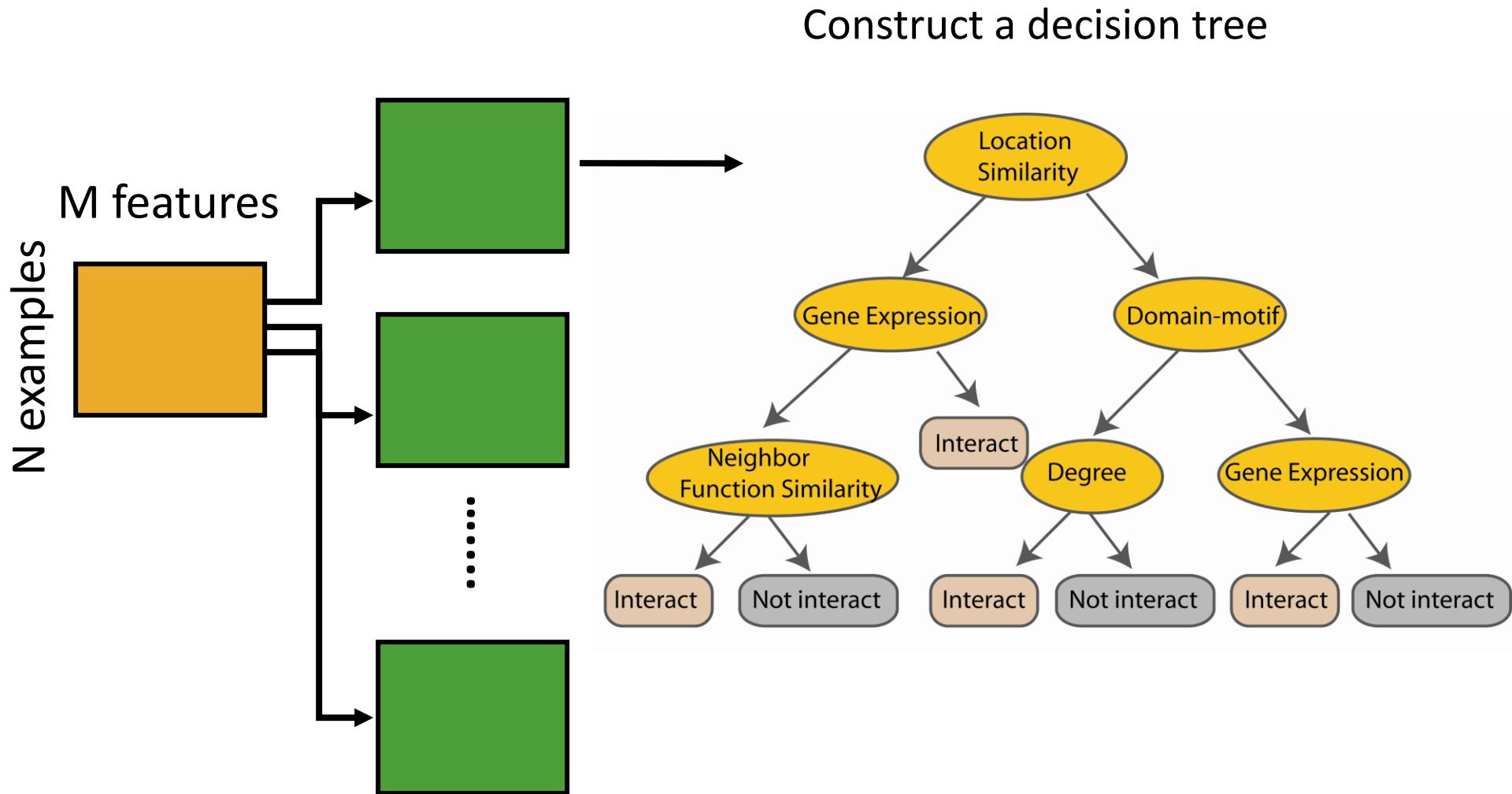
M features



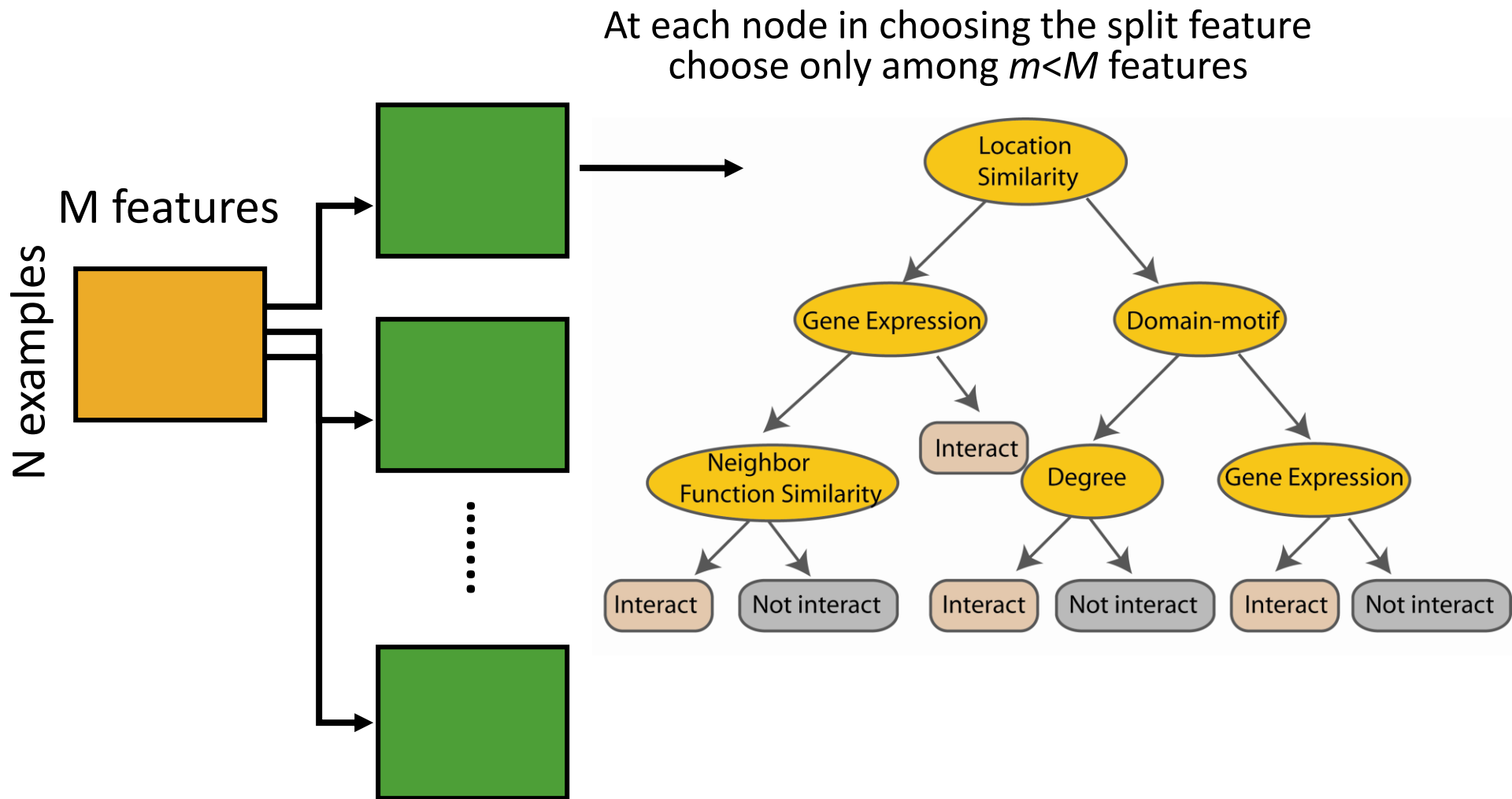
Random Forest Classifier



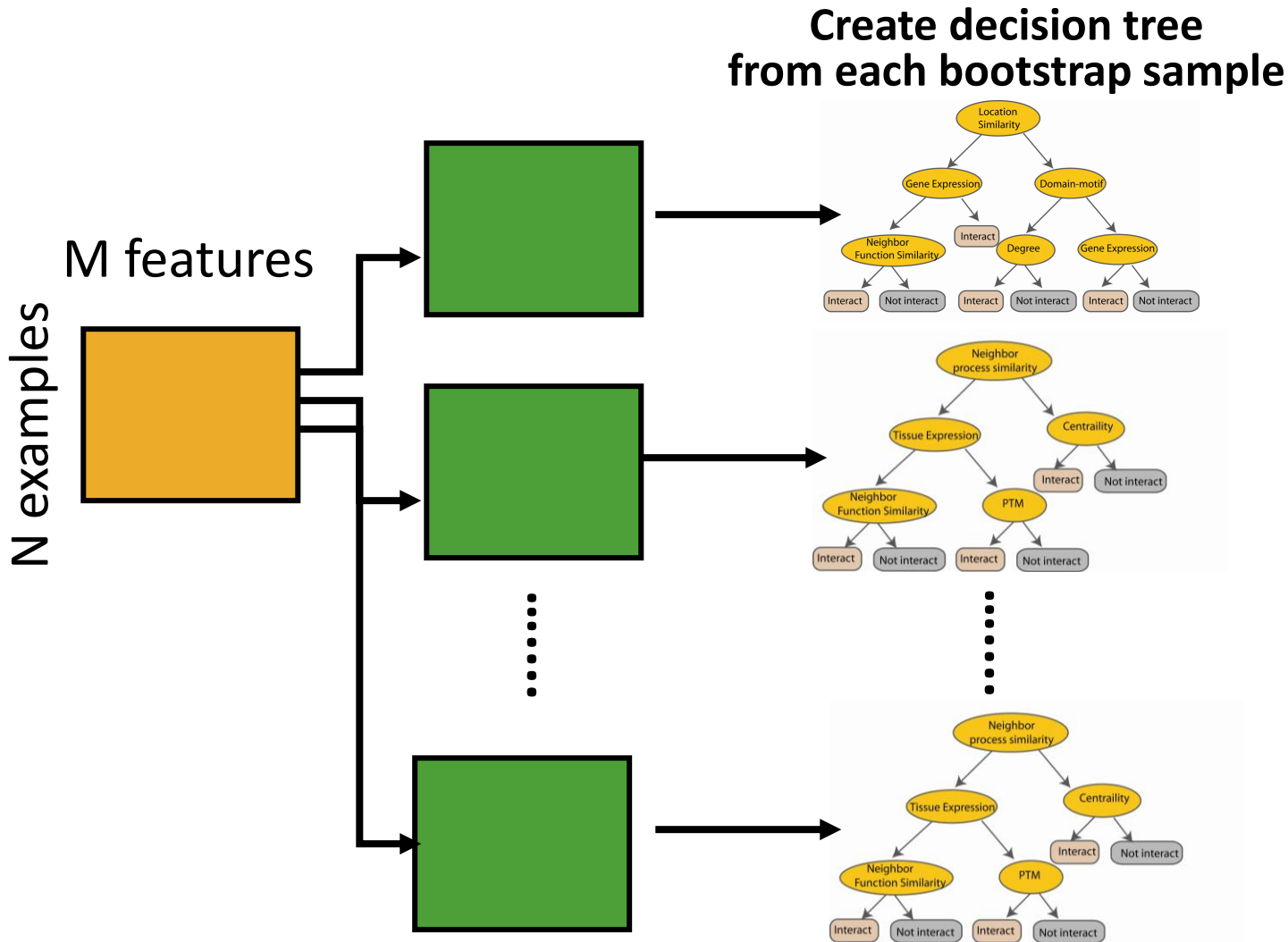
Random Forest Classifier



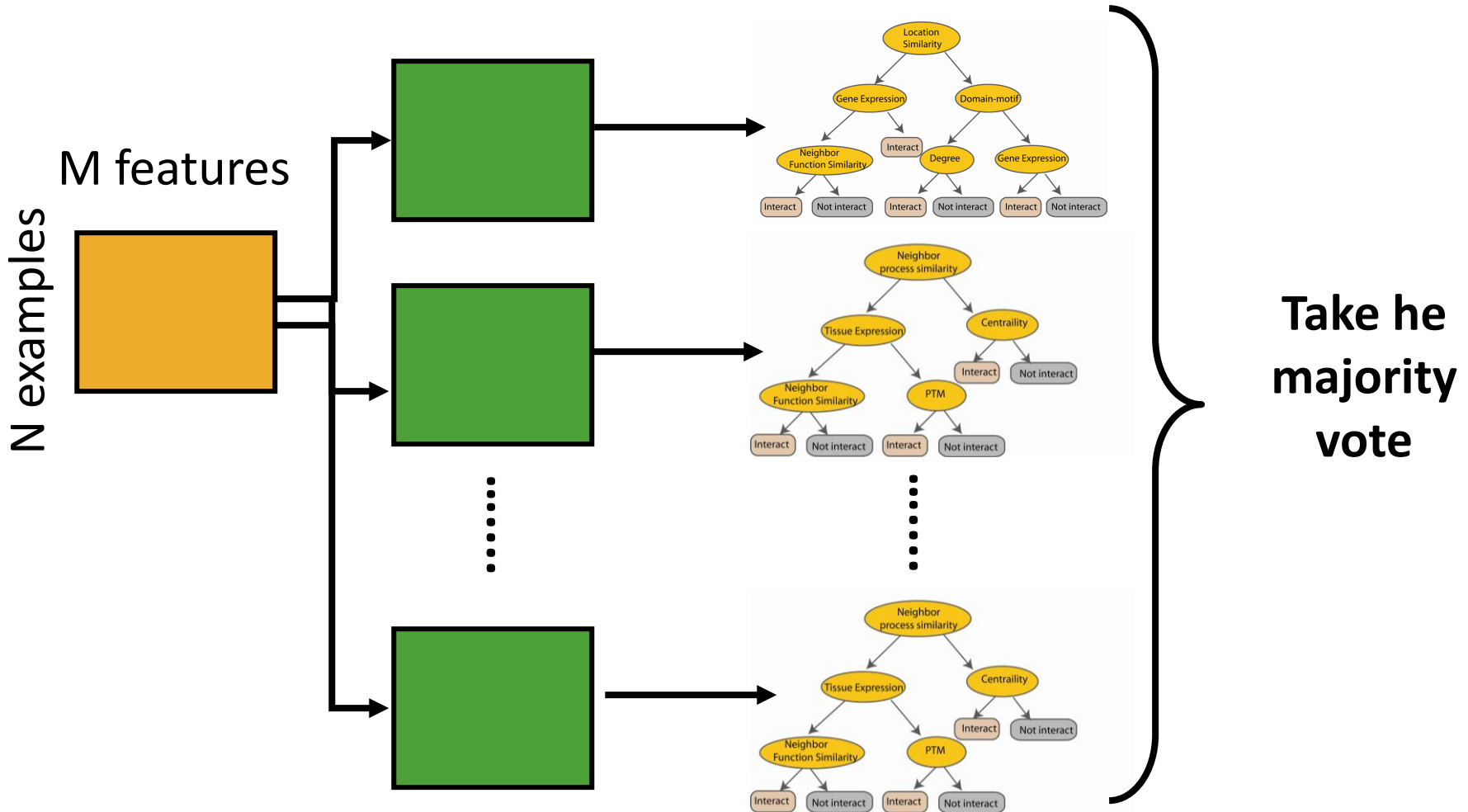
Random Forest Classifier



Random Forest Classifier



Random Forest Classifier



Strengths

- It runs efficiently on large data bases.
- It can handle thousands of input variables without variable deletion.
- It gives estimates of what variables are important in the classification.
- It has an effective method for estimating missing data and maintains accuracy when a large proportion of the data are missing.
- Generated forests can be saved for future use on other data.

Strengths

- Prototypes are computed that give information about the relation between the variables and the classification.
- The capabilities of the above can be extended to unlabeled data, leading to unsupervised clustering, data views and outlier detection.
- It offers an experimental method for detecting variable interactions.
- Random forest run times are quite fast, and they are able to deal with unbalanced and missing data.
- They can handle binary features, categorical features, numerical features without any need for scaling.
- There are lots of excellent, free, and open-source implementations of the random forest algorithm.
- We can find a good implementation in almost all major ML libraries and toolkits.

Weakness

- A weakness of random forest algorithms is that when used for regression they cannot predict beyond the range in the training data, and that they may over-fit data sets that are particularly noisy.
- The sizes of the models created by random forests may be very large. It may take hundreds of megabytes of memory and may be slow to evaluate.
- Random forest models are black boxes that are very hard to interpret